
EVALUATION OF MACHINE LEARNING ALGORITHMS FOR MESSAGE SPAM FILTERING

A PREPRINT

Ata, Heyam

Department of Computer Science
Northwest Vista College
San Antonio, TX, 78251
hata1@student.alamo.edu

 **Izzat Alsmadi**

Department of Computing and Cyber Security
Texas A&M, University
San Antonio, TX, 78224
ialsmadi@tamusa.edu

July 22, 2023

ABSTRACT

Over the years, we have expanded the incorporation of technology into our day to day lives in numerous ways such as phones and computers. These devices allow for humans to stay connected and have instantaneous communication. Not only can messages be sent in an instant but can be sent on a massive scale. This is commonly used by advertisers and what we commonly view as spam mail. With a growing presence of spam follows an increasing need for methods to combat this issue. This is the aim of many machine learning algorithms with the task of sorting through data in a specific file and identifying whether it is spam or not. This work will compile data from various machine learning algorithms and test each one's accuracy. The algorithms will be tested using a dataset containing many text messages some of which being spam and some not spam which will be referred to as ham. The accuracy score of these algorithms were output in the code when ran. Overall, this paper will dwell on spam detection and the accuracy of varying machine learning algorithms.

Keywords Machine Learning · Prediction modeling · Spam detection

1 Introduction

Identifying spam content is important as it can shield a user from harmful content, so it is crucial to know which machine learning algorithms are best suited for this task. Users typically spend a significant time daily wasted going through their email, cleaning and getting rid of spam emails. This process is not only time consuming but also creates security issues and concerns as some of those spam emails may cause users to expose sensitive or classified information creating more serious issues or problems.

All or most email systems provide spam or junk emails detection or prediction. Nonetheless, regardless of how powerful or reliable the email system, it is unlikely that such email system will make a high accurate prediction. In predicting spam emails, we typically talk about two types of errors that spam detector may produce:

- False Positive (FP), where the detector may falsely detect an email as a spam. This means that a relevant or useful email is sent to the spam or junk folder. Users who don't check spam folder may lose important information based on such errors.
- False Negative (FN), where the detector or the email system will falsely label an email as a useful or relevant email and so send it or keep it in the main mailbox. Problems of such errors is that users will waste their time going through such emails to eventually remove or delete them.

The reason why spam detectors continue to fail reaching zero FP and or zero FN is the complexity of the detection process and the several criteria that can impact such prediction. Additionally, spammers are continuously improving and updating their methods to create emails that look real or genuine.

In this work classifiers such as, variations of Naive Bayes (NB), Decision Trees (DT), ensemble classifiers, Logistic Regression (LR), K-Nearest Neighbor (KNN), Recurrent Neural Networks (RNN), and Support Vector Machine (SVM) were trained and tested. Upon receiving the results of their performance they were then compared with one another.

2 Goals and Approaches

Our main goal in this paper is to evaluate using several machine learning algorithms in the ability to detect and predict spam emails. We used a popular open spam data sets that is used by several other relevant researches. When evaluating the performance of algorithms, the data set used is a crucial component. There are many factors to take into account when selecting a data set such as size and file type. Generally speaking, a larger data set will allow for more thorough and reliable results as opposed to a small data set. In this project, a comma separated value (CSV) file was used. A CSV file allows for a great amount of data to be stored while maintaining an easy to process structure. The CSV file used (Spam.csv), contains 5573 rows of data set. Each row has two columns being used on the right, containing the messages and on the left, the message is given a label of spam or ham. Among the 5573 messages 13.41% were spam and the remaining 86.59% were ham, the classifiers will attempt to accurately predict which of the messages are spam and which are ham, 1.

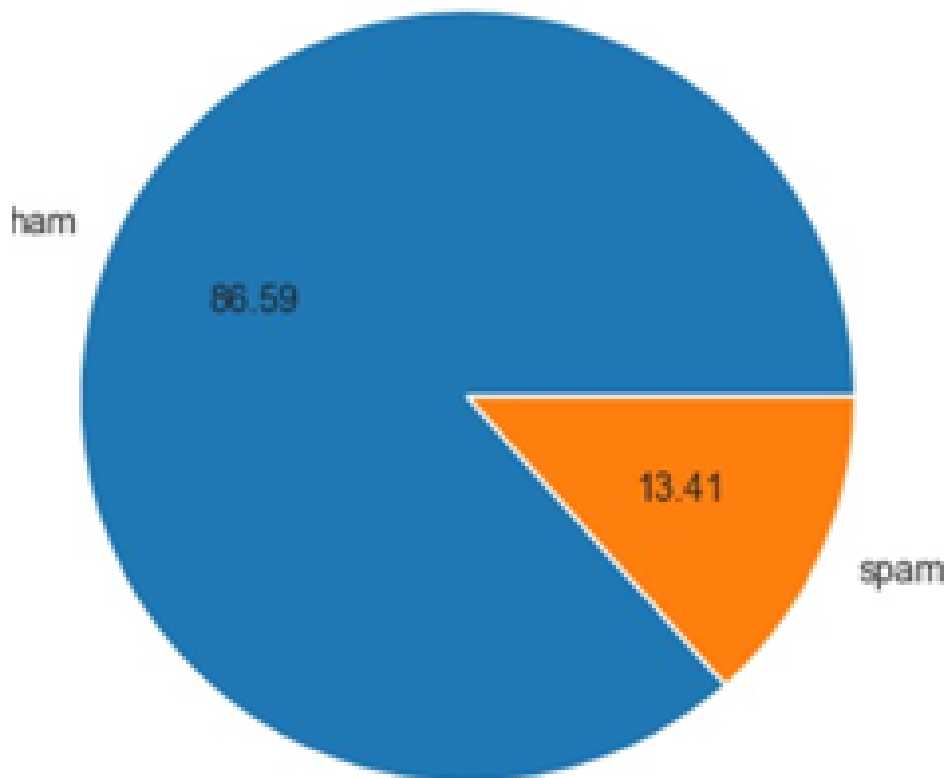


Figure 1: Percentage of ham/spam in the data set

In order for the algorithms to be trained using the data set, the data must first be split into testing and training sets. The training sets allow for the models to learn the patterns of the data, what is learned is then applied to the testing set. Of the 5573, 80% was used for training, and 20% for testing. After training the model, the testing set is used to make predictions on which text is spam and ham. These predictions are what will be used to determine a classifier's accuracy.

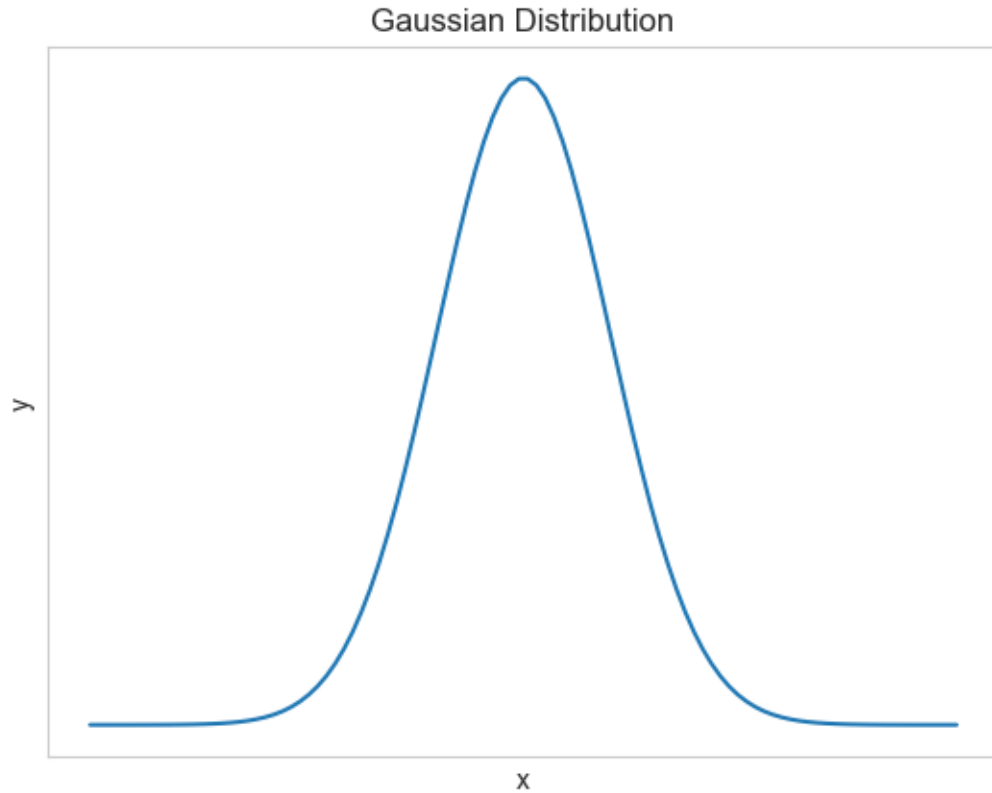


Figure 2: Gaussian Bell Curve

2.1 Naive Bayes

In this work the Naive Bayes (NB) algorithm was among the group of algorithms to be tested on this data set. The algorithm uses both Bayes theorem and an assumption of independence. Bayes theorem is an equation of probability. Specifically, the probability of something occurring based on the occurrence of another event. When applying the Naive assumption, those occurrences become independent from one another regardless of whether or not that applies. The Naive Bayes algorithm primarily consisted of three main types including Multinomial NB, Bernoulli NB, and Gaussian NB. Each of these were applied to the data set and tested.

2.1.1 Multinomial Naive Bayes

Multinomial Naive Bayes, being a variation of Naive Bayes, continues with the assumption of independence and ignores all dependencies of attributes. was specifically designed for text classification and quite commonly used in spam filtering. Multinomial NB incorporates the Naive Bayes algorithm. The features this classifier uses in order to make its decisions include the frequency of words. Among the other Naive Bayes classifiers, Multinomial performed with the greatest accuracy score of 0.98.

2.1.2 Gaussian Naive Bayes

Gaussian Naive Bayes (GNB) is another variation of Naive Bayes and follows the same parameters while adding its own. As well as an assumption of independence, GNB assumes all the features follow a distribution method known as Gaussian or normal distribution. The Gaussian distribution is a type of probability distribution. When graphed it is depicted as a bell shaped curve, with the peak being the mean in between standard deviation points.

In regards to the data set Gaussian distribution would be associated with features such as the frequency of words or the length of the message. In this work GNB utilized the Bayes theorem and assumed the variables in the data set followed Gaussian Distribution in order to calculate probability and make predictions on what is spam and what is ham. Despite

```
# ham = 0, spam = 1  
target = target.replace("ham", 0)  
target = target.replace("spam", 1)
```

Figure 3: Converting to binary values

this combination, the GNB classifier manages to do the worst in terms of accuracy in this work. It received an accuracy score of 0.9013452914798207.

2.1.3 Bernoulli Naive Bayes

Another Naive Bayes algorithm used in this work is Bernoulli Naive Bayes (BNB), this variant works well with high dimensional data such as that provided in the data set. BNB applies Naive Bayes algorithms and classifications and is a classification system specifically catered to binary and Boolean aspects. Unlike the previously mentioned Naive Bayes algorithms; Multinomial and Gaussian, Bernoulli Naive Bayes must be working with binary data in order to run. Bernoulli Naive Bayes follows a decision rule based on the below equation. Data that is evaluated by Bernoulli must follow a multivariate Bernoulli distribution. In this work our data is converted into binary values where spam is 1 and ham is 0. The relationship between the dataset and BNB classifier proved to be efficient as the classifier achieved an accuracy score of 97.67% which is a noticeable difference when comparing it to the previously mentioned Gaussian which achieved a 90.13% accuracy.

2.2 Decision Trees

In this work, two different classifiers were used that were based on decision trees. Decision trees are a type of algorithm and can be used primarily classification and regression, but also other machine learning tasks. Random Forest and Extra Trees are the variants that will be evaluated in this work.

2.2.1 Random Forest

This application of a decision tree in this work was with the Random Forest (RF) which is an ensemble classification algorithm. The method of Random Forest is training multiple decision trees of the same type, which are created using the Bagging (bootstrapping aggregating) technique. These trees are trained on randomly chosen subsets of the training sets. In this work specifically, the decision tree parameter is set 100, meaning 100 decision trees will be made. This is represented in the code with the snippet, `n_estimators= 100`. It is worth noting that altering the number of trees does in fact increase/decrease the accuracy score but the effect is not potent. 100 decision trees gave us an accuracy of 97.13%, and when 200 trees were done it gave the same result. However, when testing with 10 trees, the accuracy score went down to 96.14%.

Random Forest is commonly credited for being more robust with noisy data. This ability is due to the use of bagging. Bagging, also known as Bootstrap Aggregating, has a similar approach with using random subsets.

2.2.2 Extra Trees Classifier

Another adaptation of Decision Trees in this work includes the Extra Tree classifier. It is quite similar to RF, but it does differ in some key aspects such as how the trees are assembled. Unlike Random Forest which builds the trees using random subsets of the data and features, Extra Trees builds trees using the full training set. After the decision trees are built, a final prediction is drawn based on the average outcome of the trees.

Just as RF, the parameters for Extra Trees were set so that 100 decision trees would be made. These parameters allowed for the classifier to produce a predictive accuracy of 98.12% on the dataset.

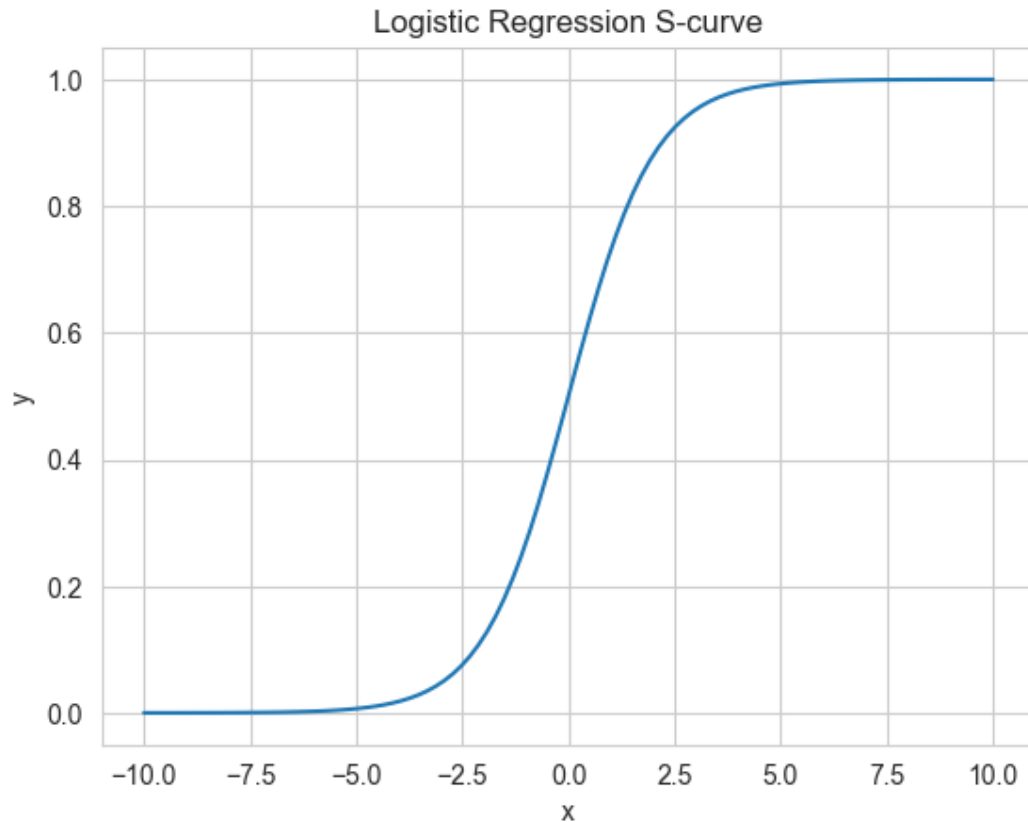


Figure 4: Logistic Regression S-Curve

2.3 Bagging

In this work we are able to see Bagging be applied to other classifiers such as Random Forest and Extra Trees. Although bagging is an effective technique to be applied to other models, it still proves to perform exceptionally as a standalone classifier.

Bagging, also known as Bootstrap Aggregating, uses a method of randomly selecting subsets of the training data. This arbitrary method allows for the possibility of data being used more than once or not at all. These subsets are used to train different classifiers and how many classifiers is dependent on the parameters set, in this work the parameters were set to 100, meaning 100 classifiers. As these models are trained they make predictions which are all combined to form the final predictions which is what is ultimately outputted by the Bagging method. These predictions provided an accuracy of 96%. Although a .96 accuracy score is fairly high, this was actually the lowest yielded score.

2.4 Logistic Regression

Logistic Regression (LR) is a statistical machine learning classification model which has a primary purpose of text classification. It specifically works with binary values, choosing between two categories. Its predictions will ultimately be true or false and in this case spam or ham. However to come to this conclusion the model needs to make the predictions based on the given data such as certain characteristics. In this code word frequency is what is used. In addition to word frequency, LR is using word association and how strong the presence or absence of a word is to the message.

When graphed Logistic Regression will present as an S-curve. The x-axis will represent the given data or characteristic while the y axis will have the probability of the message being spam. In this code the use of a Logistic Regression classification model was able to output an accuracy of 0.9767 in predicting which messages were spam.

2.5 K-Nearest Neighbor Classifier

K-Nearest Neighbor (KNN), is an algorithm that can be used for both regression and classification, which in this work, the algorithm was used for classification purposes. The method of KNN is based on the already existing information. KNN will classify a message as spam based on what has already been deemed as spam. It is important to note that the existing classifications are from the training data and the algorithm is simply storing them to apply that info to the testing set. If message A is what the algorithm is trying to classify, the algorithm will go based on the other nearest points. The distance of a point is not physical and is calculated based on differences in characteristics, the similarities present the nearer. Depending on the parameters set such as, $k=1$ or by default $k=5$ the prediction can be different. If $k=1$ then the algorithm will only make the prediction based on a single nearest point. In this work, the default $k=5$ was used which considers the 5 nearest points and uses what the majority class is to make the prediction on message A.

2.6 Gradient Boosting

Gradient Boosting (GB) is another machine learning algorithm included that can be used for both regression and classification. Gradient Boosting is similar to Bagging in the sense that both are ensemble learning methods and use numerous models to make their prediction. GB makes predictions by creating weak models after one another, which is where it differs from Bagging as Bagging creates models in no particular order. Each GB model tries to correct the mistakes of the previous model. This continues until it has reached the amount set, which in this work was 100, once the 100 models were created the algorithm was able to make final predictions. This process of creating classifiers in a sequential order to improve on one another is called Boosting. Overall this algorithm was able to classify the text with an accuracy score of .9641.

2.7 AdaBoost

Adaptive Boosting, primarily known as AdaBoost, just as GB uses a Boosting algorithm. In this code we have used AdaBoost with decision trees. AdaBoost creates these weak variations of decision trees and continues to make more and more in a sequential order. It also has the same parameters as GB regarding the amount of classifiers being made. Each new version of the classifier is a revision of the previous one which implies the next one performs better. When handling errors, AdaBoost has a unique approach. The algorithm involves assigning the mistakes a higher weight in order to prioritize them in the next classifier. This is what allows for the improvements to be made on the overall model. This method was adequate for the task for spam detection as it accuracy predicted at a rate of 97.67%.

2.8 Support Vector Machine

In this work, we included a Support Vector Machine (SVM). SVM has the ability to be used for both regression and classification purposes, however we are using it for text classification. In this work we selected a linear kernel for SVM. A linear kernel is used in the case of two groups needing separation. In regards to this project the two groups would be spam and ham, categories of which the given data set will be separated into.

In terms of a graph, SVM can be described best with a scatter plot. The plots are one of two groups and the SVM decides where the best location for a boundary is in order to separate the two groups. SVM produced quite fair results scoring a 97.85% in terms of accuracy.

2.9 Recurrent Neural Network

Within the Neural Networks class are Recurrent Neural Networks (RNN). Also programmed to work with sequential data, which pair well for the use in this project (spam detection). A fundamental component to a RNN is its Hidden State, this is the algorithm remembering previous input. This is also known as the Memory State. The Hidden State is updated at each step and it remembers information regarding the sequence. Also, depending on the architecture, the algorithm works in different ways. However in this project a sequential architecture was used. The algorithm goes through multiple steps to draw a conclusion, but at each step it makes a decision and uses them to come to a final conclusion. The network has shown to uphold in this work when predicting what data is spam and what is not by achieving an accuracy score of 98.39%.

3 Results and Analysis

Overall, of the 12 algorithms to be tested majority proved to sufficient enough to handle the data set. None of the Algorithms accuracy's were below 90%. In fact, all but one were in range of .96-.98. However, the Gaussian Naive

Classifier	Accuracy	Precision	F1
Multinomial Naive Bayes	.98	.96	.95
Gaussian Naive Bayes	.90	.96	.95
Bernoulli Naive Bayes	.97	.96	.95

Figure 5: Naive Bayes: Result Comparison

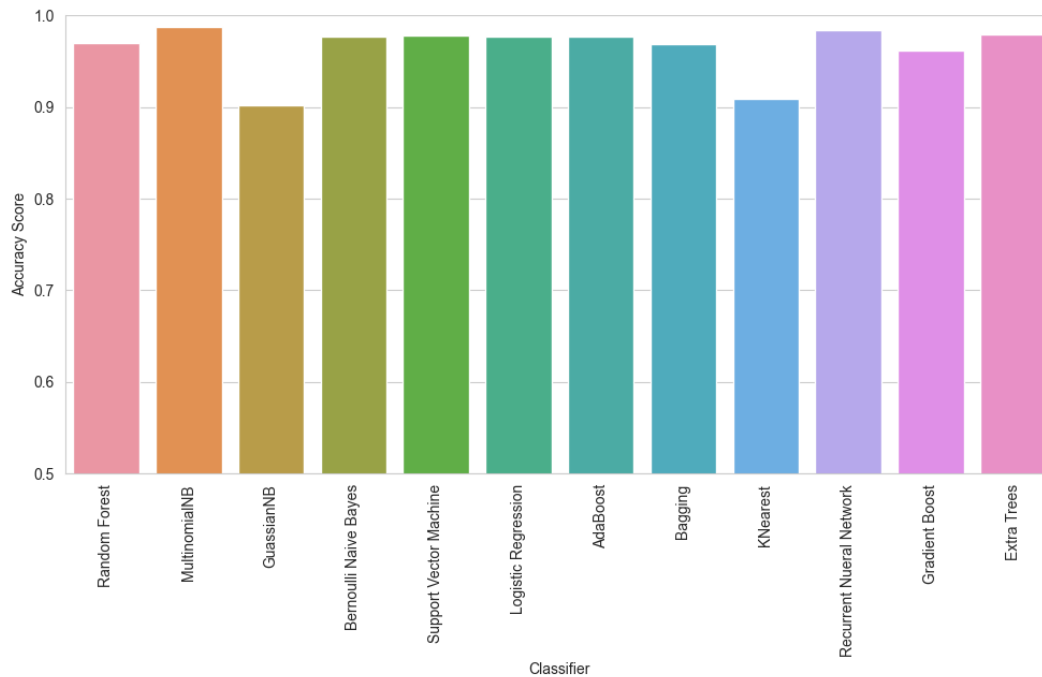


Figure 6: Accuracy of Algorithms

Bayes approach received the lowest accuracy score at, 0.9013. This result does not reflect the entirety of the Naive Bayes umbrella as the best performing algorithm with the data set was Multinomial Naive Bayes and received a .9874 accuracy score. Following MNB would be Keras Sequential Model producing a .9857 accuracy score when predicting if the texts on the data set were spam and not.

Some other notable algorithms were those with decision trees. Decision trees were used in different manners such as Random Forest and Extra Trees. The two classifiers both had the same parameters set but took different routes in building their decision trees. This contrast is what produced their differing results where Random forest achieved a 97% accuracy and Extra Trees a 98%, performing slightly better. Extra Trees was able to perform better in this text classification due to approach on randomization and its ability to work in robust environments.

Boosting was also represented in this work and in more easy than one such as Gradient Boosting and Adaptive Boosting also known as AdaBoost. These two algorithms performed exceptionally with Gradient Boost performing with the accuracy score of 96.4% and AdaBoost with 97.8%. Adaptive Boosting is likely to have received a higher accuracy score due to how it handles mis-classifications. It's routine of assigning a higher weight to those mis-classified allows for a reduction in error.

Author (last name)	Models and Accuracies (%)				
	NB	SVM	KNN	Adaboost	RF
Ata & Alsmadi	98.74	97.84	96.41	97.67	97.22
Mehr	98.88	98.86	97.47	98.59	98.57
Sheneamer	81.02	85.46	**	**	88.14
Yaseen	94.69	**	92.92	**	**
Awad	99.46	96.9	96.2	**	**

Figure 7: Comparing Accuracy

4 A comparison Study

A previously done project on SMS spam detection was done by Housemand Mehr, Shirani-Mehr [2013]. Their work applied different machine learning algorithms onto a provided spam collection data set. The data set similar to the one used in this work, contained both spam and non spam messages. The presence of spam was extremely similar in their data set as it was 13.40% spam and in this work the data set was compiled of 13.41% spam. The algorithms tested on the data set included, Random Forest, Multinomial Naive Bayes, Support Vector Machine, K-nearest Neighbor, and Adaboost. Mehr’s study yielded a result showing Multinomial Naive Bayes to be the best performing algorithm with detecting SMS spam. In this work Multinomial Naive Bayes shared the same result and outperformed all test algorithms. K-nearest Neighbor directly followed Multinomial Naive Bayes in this work and received the second best accuracy. However, in Mehr’s work SVM was instead second best performing. In both projects SVM was used with a linear Kernel. However, in their work they expanded beyond that and tried more kernel such as Polynomial and Sigmoid. Despite the expansion of different kernels, linear appeared to perform the best. Although our work also used a linear kernel, it did not perform as well as the comparison and had a slightly greater error margin.

Many varying works were done surrounding spam detection. Another one, which was done by Abdullah Sheneamer, Sheneamer [2021]. His study utilized multiple datasets in combination totaling to 16,872 not-spam and 5,243 spam SMS messages, that is a total of 76.3% being not-spam. The dataset is significantly larger than the one used in this paper. His work tested and compared the capabilities of several algorithms.

The next paper, “Email Spam Detection Using Deep Learning Techniques,” authored by Qussai Yaseen, Yaseen et al. [2021] is yet another project indulge into different algorithms. The algorithms tested included Naive Bayes and K-nearest Neighbor. The dataset utilized is notably small with only 226 samples and 50% being spam. They also used 70% of the sample as testing data. These parameters yielded results of a 94.69% accuracy for Naive Bayes and a 92.92% for K-Nearest Neighbor.

Alsmadi and Alhami paper presented also different classification models on email spam detection, Alsmadi and Alhami [2015]. Dataset was extracted personally from authors emails systems. In addition to prediction and classification, they also presented different methods to auto cluster emails based on subjects or categories.

Lastly, Awad paper, Awad and ELseuofi [2011] targetted also email spam detection, and test different classifiers including, Naive Bayes, Support Vector Machine, K-nearest Neighbor, Neural Networks, and more. They extracted a dataset from the SpamAssassin website, which contains 6000 emails, 37.04% being spam. The percentage was reused when determining the size of the testing set which was 37.04% leaving 62.96% of the emails for training. Their best performing classifier in terms of accuracy was Naive Bayes, yielding a score of 99.46%. This is the highest overall, score across all papers.

In the table below is a side by side comparison of how the different machine learning methods performed in different datasets. We will use the authors’ last names to distinguish the different works. Two asterisks (**) will be in place of an accuracy in the case that the paper did not include that specific algorithm.

5 Conclusion

In this work several machine learning algorithms were tested on a spam data set with the purpose of classifying which is spam and which is not. The results indicated that a Multinomial Naive Bayes approach worked best with this task and with this specific data set. MultinomialNB produced the highest accuracy on all tested algorithms. The test algorithms used many features of the text such as length, characters and the frequency of features in order to produce results.

The results provided insight on how these different algorithms reacted in the given environment: the spam riddled data set. It is seen how some algorithms performed better than others when working with this data set.

References

Houshmand Shirani-Mehr. Sms spam detection using machine learning approach. *unpublished*) <http://cs229.stanford.edu/proj2013/ShiraniMehr-SMSSpamDetectionUsingMachineLearningApproach.pdf>, 2013.

Abdullah Sheneamer. Comparison of deep and traditional learning methods for email spam filtering. *International Journal of Advanced Computer Science and Applications*, 12(1), 2021.

Qussai Yaseen et al. Spam email detection using deep learning techniques. *Procedia Computer Science*, 184:853–858, 2021.

Izzat Alsmadi and Ikdam Alhami. Clustering and classification of email contents. *Journal of King Saud University-Computer and Information Sciences*, 27(1):46–57, 2015.

Wael Abou Awad and SM ELseuofi. Machine learning methods for spam e-mail classification. *International Journal of Computer Science & Information Technology (IJCSIT)*, 3(1):173–184, 2011.

<https://cs229.stanford.edu/proj2013/ShiraniMehr-SMSSpamDetectionUsingMachineLearningApproach.pdf>

https://www.researchgate.net/profile/Sanket-Sonowal/publication/344050184_Email_Spam_Detection_Using_Machine_Learning_Algorithms_Spam_-_Detection_-_Using_-_Machine_-_Learning_-_Algorithms.pdf

<https://d1wqtxts1xzle7.cloudfront.net/55309752/10.1.1.190.1441-libre.pdf?1513496812=response-content-disposition=inlinehttps://pdfs.semanticscholar.org/d319/43a104227f0e56cf97e73914010c96cc50c7.pdf>

https://www.researchgate.net/profile/J-E-T-Akinsola/publication/318338750_Supervised_Machine_Learning_Algorithms_Classification_Machine_-_Learning_-_Algorithms_-_Classification_-_and_-_Comparison.pdfhttps://journalofbigdata.springeropen.com/articles/10.1186/s40537-015-0029-9